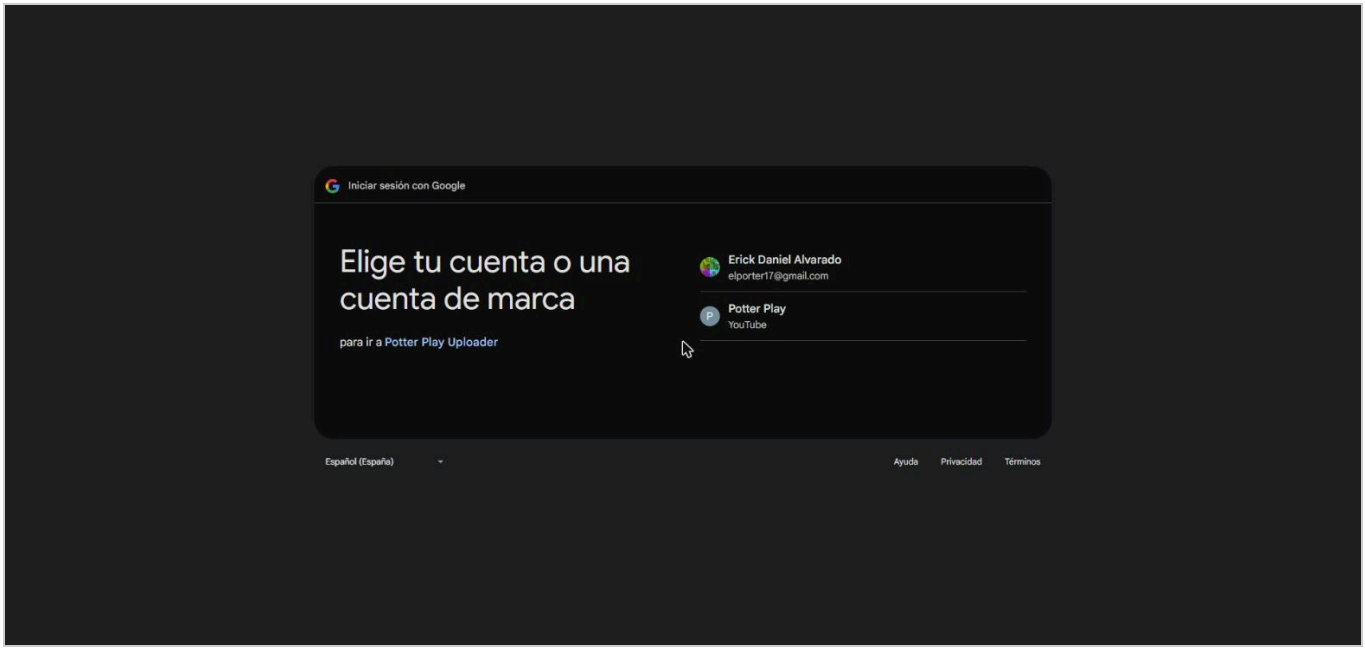
Potter Play Uploader · flujo de OAuth, script completo de subida y resultado

Proyecto de Google Cloud **potter-play-youtube** (n.º 170439840289). Canal: **Potter Play** (https://www.youtube.com/@potterplay). Herramienta interna de un solo creador; código público: https://github.com/elporter17/estudio-epiciclos

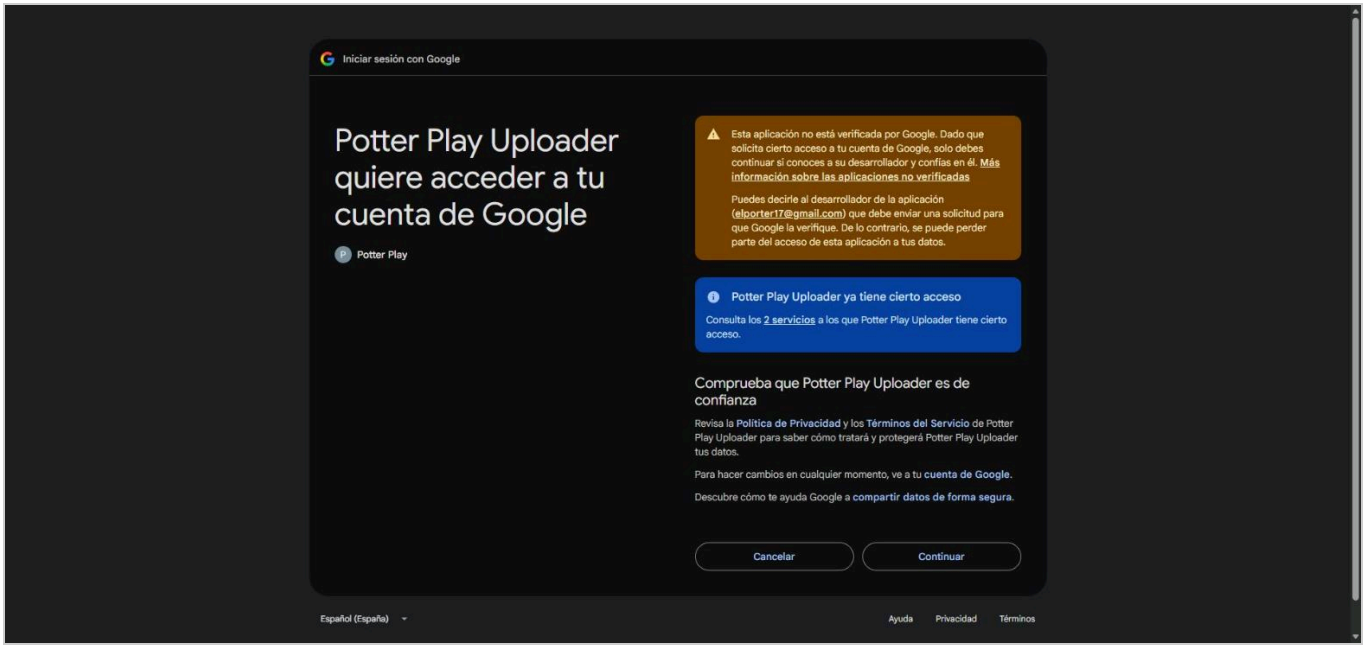
Proceso de principio a fin

1. El dueño del canal autoriza una sola vez con `npm run youtube:connect` (OAuth 2.0 de Google; permisos youtube.upload y youtube.force-ssl). El token de renovación se guarda cifrado en los secretos de GitHub.
2. Un programador lanza GitHub Actions; el script dibuja el video (contenido propio: canvas + ffmpeg, música propia) y llama a `subirYouTube()`, que hace una subida reanudable a `videos.insert` con `privacyStatus: "private"`.
3. El aviso de la corrida trae el enlace de YouTube Studio. El dueño revisa el video y lo publica a mano.
4. La cuota se limita en el propio código: como máximo 5 subidas al día.
5. El acceso se revoca en myaccount.google.com → Aplicaciones vinculadas → Potter Play Uploader → Eliminar.

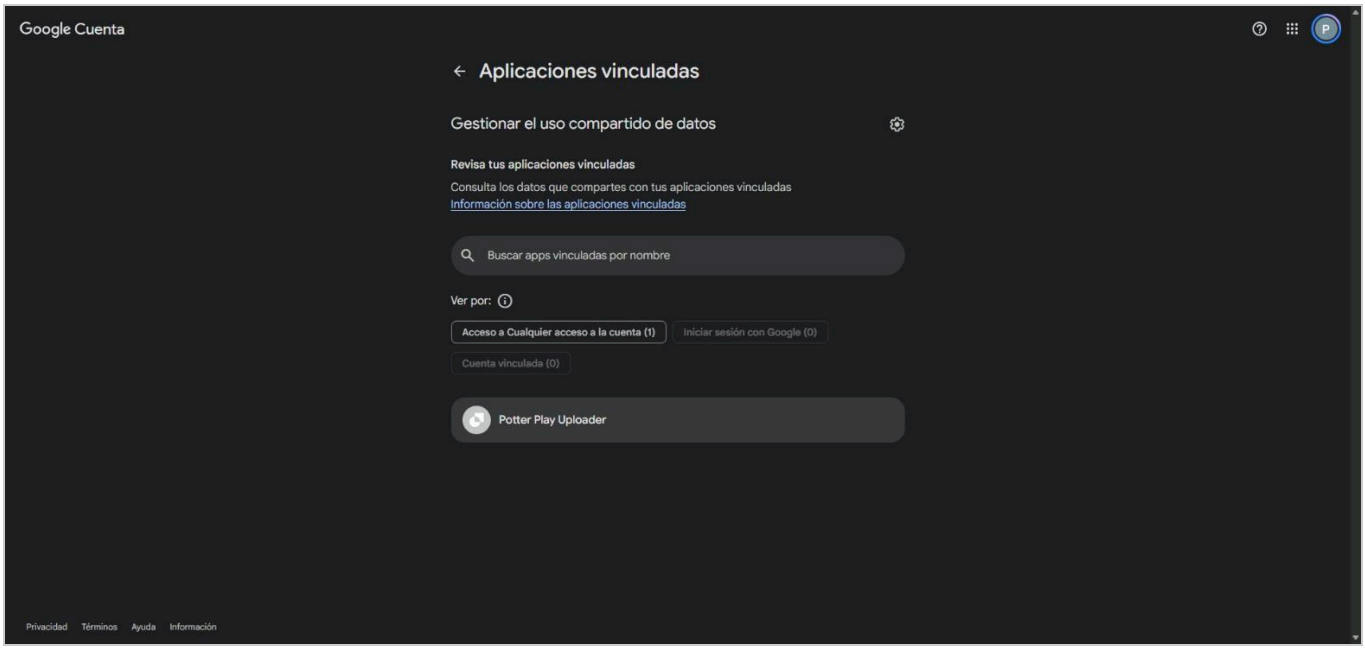
a) OAuth 1: el dueño elige el canal (cuenta de marca «Potter Play»)



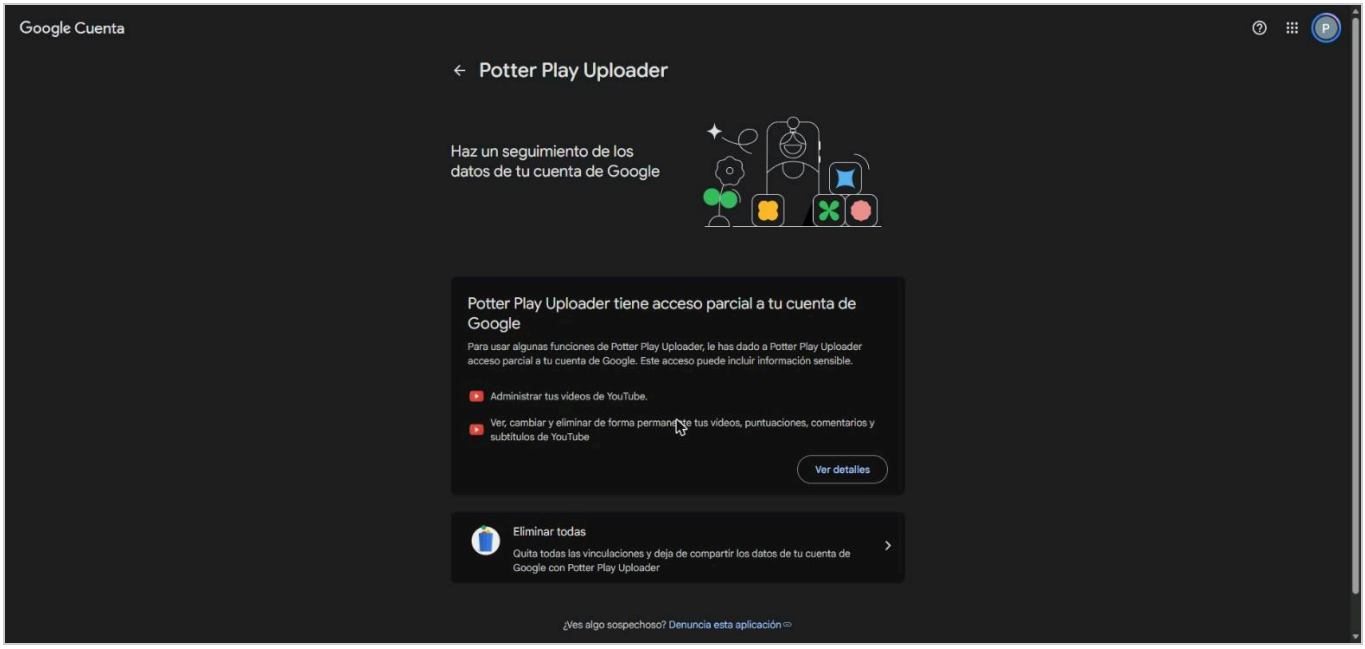
a) OAuth 2: pantalla de consentimiento de «Potter Play Uploader»



a) OAuth 3: la app aparece en «Aplicaciones vinculadas» de la cuenta



a) OAuth 4: permisos concedidos y opción de revocar («Eliminar todas»)



b) scripts/lib/youtube.mjs (biblioteca de subida completa)

```
// YouTube Data API v3 (proyecto de Google Cloud «potter-play-youtube», cliente de escritorio «Subidas Potter Play»).
// Credenciales: YOUTUBE_CLIENT_ID, YOUTUBE_CLIENT_SECRET y YOUTUBE_REFRESH_TOKEN (sale de «npm run youtube:connect»),
// en .env y en los secretos de GitHub. Los videos se suben PRIVADOS (como borrador): se publican a mano desde Studio.
// Mientras Google no audite el proyecto, YouTube deja privado todo lo que se sube por API.
// Cuota: 10.000 unidades al día; cada subida cuesta 1.600 (≈ 6 al día), un comentario 50.
import fs from 'node:fs';

export const SCOPES = ['https://www.googleapis.com/auth/youtube.upload', 'https://www.googleapis.com/auth/youtube.force-ssl'];
const API = 'https://www.googleapis.com/youtube/v3';

export const youtubeReady = () => !!(process.env.YOUTUBE_CLIENT_ID && process.env.YOUTUBE_CLIENT_SECRET &&
process.env.YOUTUBE_REFRESH_TOKEN);

export async function tokenDesdeCodigo(code, redirect_uri) {
  const r = await fetch('https://oauth2.googleapis.com/token', {
    method: 'POST', headers: { 'content-type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({ code, redirect_uri, grant_type: 'authorization_code', client_id: process.env.YOUTUBE_CLIENT_ID,
client_secret: process.env.YOUTUBE_CLIENT_SECRET })
  });
  const j = await r.json(); if (!r.ok) throw new Error(`Google: ${j.error_description || j.error || r.status}`);
  return j;
}

let cache = null;
export async function accessToken() {
  if (cache && cache.vence > Date.now() + 60e3) return cache.token;
  const r = await fetch('https://oauth2.googleapis.com/token', {
    method: 'POST', headers: { 'content-type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({ grant_type: 'refresh_token', refresh_token: process.env.YOUTUBE_REFRESH_TOKEN, client_id:
process.env.YOUTUBE_CLIENT_ID, client_secret: process.env.YOUTUBE_CLIENT_SECRET })
  });
  const j = await r.json(); if (!r.ok) throw new Error(`Google no renovó el token: ${j.error_description || j.error || r.status}`);
  cache = { token: j.access_token, vence: Date.now() + j.expires_in * 1000 };
  return cache.token;
}

export async function api(ruta, { method = 'GET', query = {}, body } = {}) {
  const t = await accessToken(), url = `${API}${ruta}?${new URLSearchParams(query)}`;
  const r = await fetch(url, { method, headers: { authorization: `Bearer ${t}`, ...(body ? { 'content-type': 'application/json' } : {}) },
body: body ? JSON.stringify(body) : undefined });
  const j = r.status === 204 ? {} : await r.json().catch(() => ({}));
  if (!r.ok) throw new Error(`YouTube ${ruta}: ${j.error?.message || r.status}`);
  return j;
}

export const miCanal = () => api('/channels', { query: { part: 'snippet,statistics', mine: 'true' } }).then((j) => j.items?.[0] || null);

// Título: máximo 100 caracteres; descripción: 5.000; etiquetas: 500 en total. Shorts: vertical y de hasta 3 min
// (YouTube lo detecta solo; #Shorts en el título ayuda). categoryId 27 = Educación, 24 = Entretenimiento.
export async function subirYouTube({ file, titulo, descripcion = '', etiquetas = [], privacidad = 'private', categoria = '27', publicarEn })
{
  const t = await accessToken(), size = fs.statSync(file).size;
  const tags = []; let n = 0; for (const e of etiquetas) { const x = String(e).replace(/[<>#]/g, '').trim(); if (x && n + x.length + 1 <=
480) { tags.push(x); n += x.length + 1; } }
  const meta = {
```

```
    snippet: { title: titulo.slice(0, 100), description: descripcion.slice(0, 4900), tags, categoryId: categoria, defaultLanguage: 'es',
defaultAudioLanguage: 'es' },
    status: { privacyStatus: publicarEn ? 'private' : privacidad, ...(publicarEn ? { publishAt: publicarEn } : {}), embeddable: true }
  };
  // Subida reanudable: primero los metadatos (devuelve la URL de subida) y después el archivo entero
  const ini = await fetch('https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status', {
    method: 'POST', headers: { authorization: `Bearer ${t}`, 'content-type': 'application/json; charset=UTF-8', 'x-upload-content-length':
String(size), 'x-upload-content-type': 'video/mp4' }, body: JSON.stringify(meta)
  });
  if (!ini.ok) { const j = await ini.json().catch(() => ({})); throw new Error(`YouTube no aceptó la subida: ${j.error?.message ||
ini.status}`); }
  const destino = ini.headers.get('location');
  for (let intento = 1; ; intento++) {
    const r = await fetch(destino, { method: 'PUT', headers: { authorization: `Bearer ${t}`, 'content-type': 'video/mp4', 'content-length':
String(size) }, body: fs.readFileSync(file) });
    const j = await r.json().catch(() => ({}));
    if (r.ok) return { id: j.id, url: `https://youtu.be/${j.id}`, studio: `https://studio.youtube.com/video/${j.id}/edit`, estado:
j.status?.privacyStatus, subida: j.status?.uploadStatus };
    if (intento >= 3 || r.status < 500) throw new Error(`YouTube falló al subir: ${j.error?.message || r.status}`);
    await new Promise((ok) => setTimeout(ok, 3000 * intento));
  }
}
```

b) scripts/youtube-connect.mjs (autorización OAuth)

```
#!/usr/bin/env node
// Conecta el canal de YouTube (Potter Play): abre el navegador, eliges la cuenta y el canal, autorizas, y el token de
// renovación queda en .env como YOUTUBE_REFRESH_TOKEN (con --github, también en los secretos del repo, sin mostrarlo).
//   npm run youtube:connect                → conectar
//   npm run youtube:connect -- --github    → conectar y guardar también en GitHub
//   npm run youtube:connect -- --status    → qué canal está conectado
import fs from 'node:fs';
import http from 'node:http';
import path from 'node:path';
import crypto from 'node:crypto';
import { spawn, spawnSync } from 'node:child_process';
import './lib/env-file.mjs';
import { parseCli } from './lib/cli.mjs';
import { ROOT } from './lib/node-env.mjs';
import { SCOPES, tokenDesdeCodigo, miCanal } from './lib/youtube.mjs';

const o = parseCli({ github: { type: 'boolean', default: false }, status: { type: 'boolean', default: false } });
if (!process.env.YOUTUBE_CLIENT_ID || !process.env.YOUTUBE_CLIENT_SECRET) { console.error('Faltan YOUTUBE_CLIENT_ID y YOUTUBE_CLIENT_SECRET
en .env'); process.exit(1); }
if (o.status) {
  if (!process.env.YOUTUBE_REFRESH_TOKEN) { console.log('Sin canal conectado'); process.exit(0); }
  const c = await miCanal(); console.log(c ? `Canal: ${c.snippet.title} (${c.snippet.customUrl || c.id}) · ${c.statistics.subscriberCount}
suscriptores` : 'La cuenta no tiene canal');
  process.exit(0);
}

const state = crypto.randomBytes(16).toString('hex');
const server = http.createServer();
await new Promise((ok) => server.listen(0, '127.0.0.1', ok));
const redirect = `http://127.0.0.1:${server.address().port}`;
const url = 'https://accounts.google.com/o/oauth2/v2/auth?' + new URLSearchParams({
  client_id: process.env.YOUTUBE_CLIENT_ID, redirect_uri: redirect, response_type: 'code', scope: SCOPES.join(' '),
  access_type: 'offline', prompt: 'consent select_account', state
});
console.log(`Abre esta URL (se abre sola), elige la cuenta y el canal «Potter Play» y autoriza:\n\n${url}\n`);
if (process.platform === 'win32') spawn('cmd', ['/c', 'start', '', url.replace(/&/g, '^&')], { detached: true, stdio: 'ignore' });

const code = await new Promise((ok, mal) => {
  server.on('request', (req, res) => {
    const q = new URL(req.url, redirect).searchParams;
    if (!q.get('code') && !q.get('error')) { res.end(); return; }
    res.writeHead(200, { 'content-type': 'text/html; charset=utf-8' });
    res.end(q.get('error') ? `<h2>No se autorizó: ${q.get('error')}</h2>` : '<h2>Listo: Potter Play quedó conectado. Ya puedes cerrar esta
pestaña.</h2>');
    if (q.get('state') !== state) mal(new Error('El state no coincide')); else if (q.get('error')) mal(new Error(q.get('error'))); else
ok(q.get('code'));
  });
  setTimeout(() => mal(new Error('Pasaron 10 minutos sin autorizar')), 600e3);
}).finally(() => server.close());

const tok = await tokenDesdeCodigo(code, redirect);
if (!tok.refresh_token) throw new Error('Google no devolvió el token de renovación');
const envFile = path.join(ROOT, '.env');
let env = fs.readFileSync(envFile, 'utf8'); const nl = env.includes('\r\n') ? '\r\n' : '\n';
env = /^YOUTUBE_REFRESH_TOKEN=.*$/m.test(env) ? env.replace(/^YOUTUBE_REFRESH_TOKEN=.*$/m, `YOUTUBE_REFRESH_TOKEN=${tok.refresh_token}`) :
env.replace(/\s*$/, '') + nl + `YOUTUBE_REFRESH_TOKEN=${tok.refresh_token}` + nl;
fs.writeFileSync(envFile, env);
process.env.YOUTUBE_REFRESH_TOKEN = tok.refresh_token;
const c = await miCanal();
console.log(c ? `Conectado: ${c.snippet.title} (${c.snippet.customUrl || c.id})` : 'Autorizado, pero esa cuenta no tiene canal: vuelve a
conectar eligiendo «Potter Play»');
if (o.github) {
  for (const k of ['YOUTUBE_CLIENT_ID', 'YOUTUBE_CLIENT_SECRET', 'YOUTUBE_REFRESH_TOKEN']) {
    const r = spawnSync('gh', ['secret', 'set', k], { input: process.env[k], encoding: 'utf8', cwd: ROOT });
    console.log(r.status === 0 ? `GitHub: ${k} guardado` : `GitHub: no se pudo guardar ${k} (${String(r.stderr).trim()})`);
  }
}
```

b) scripts/curiosidades-video.mjs (dónde se llama la subida)

```
// YouTube: el mismo video (nuestra música y el narrador) como Short PRIVADO, para publicarlo a mano desde Studio
    if (o.youtube !== 'no' && youtubeReady() && cupoYouTube(pieza.formato)) {
      try {
        meta.youtube = await subirYouTube({ file: video, titulo: `${pieza.titulo} #Shorts`, descripcion: `${pieza.texto}\n\n🔔 Suscríbete a
Potter Play: videos nuevos todos los días.`, etiquetas: [...new Set([...(pieza.texto.match(/#[\p{L}\d_]+/gu) || []).map((x) => x.slice(1)),
'curiosidades', 'shorts', 'potter play'])] });
        usarCupoYouTube(pieza.formato); console.error(`YouTube: subido como privado (borrador) · ${meta.youtube.studio}`);
      } catch (e) { meta.youtube = { status: 'error', error: e.message }; console.error(`Aviso: YouTube: ${e.message}`); }
    }
```

b) Resultado: el video subido por la API, revisado y publicado en YouTube Studio

